FORMATO DE GUÍA DE PRÁCTICA DE LABORATORIO / TALLERES / CENTROS DE SIMULACIÓN – PARA DOCENTES

CARRERA: Maestría de Software		**ASIGNATURA: Patrones de Diseño de APIs**
NRO. PRÁCTICA:	03	**TÍTULO PRÁCTICA**: Stack MEAN

OBJETIVO : Diseñar e implementar una arquitectura web desacoplada, robusta y escalable utilizando el **Stack MEAN (MongoDB, Express, Angular, Node.js)** sobre **TypeScript**.

INSTRUCCIONES	**1.** Desacoplamiento del Backend y Persistencia Inversa (Patrón Repository)
	2. Blindaje de API, Validación Estricta y Patrón de Respuesta Universal
	3. Arquitectura Reactiva y Manejo de Estado en el Frontend (Angular)

Sistema de Gestión de Empleados (Módulo CRUD)

El dominio de la práctica consiste en el backend para la administración de personal de una organización. El estudiante deberá aplicar patrones de diseño avanzados, separación estricta de responsabilidades, tipado seguro y programación reactiva, transformando un CRUD tradicional en un sistema de nivel empresarial.

El sistema debe operar bajo cinco operaciones fundamentales, expuestas mediante contratos API REST semánticos:

- Consulta paginada/completa de empleados.
- Registro con validación estructural de identidad.
- Consulta atómica por identificador único (`ID`).
- Actualización parcial o total de registros mediante mutaciones seguras.
- Eliminación física o lógica mediante identificadores correlativos.

Infraestructura Tecnológica Requerida

- Node.js (LTS v20+ o superior) y npm
- Angular CLI (v16+ o superior)
- TypeScript (v5+ o superior)
- MongoDB Server / MongoDB Atlas & MongoDB Compass
- Cliente REST (Insomnia, Postman o REST Client para VS Code)
-

Código Inicial

El Desafío Académico: 4 Retos de Refactorización

Los estudiantes deberán tomar el código fuente inicial (el cual viola principios de diseño como responsabilidad única y encapsulamiento) y resolver incrementalmente los siguientes hitos de ingeniería:

[Reto 1] Desacoplamiento del Backend y Persistencia Inversa

Problema en la Base: Los controladores de Express conocen e instancian directamente el modelo de Mongoose (`EmployeeModel`), acoplando la red con el motor de base de datos.

Refactorización Exigida: Implementar el **Patrón Repository** mediante una interfaz abstracta (`IEmployeeRepository`). La capa de Express debe ser agnóstica al ODM; si se remueve el `import mongoose` del controlador, el sistema debe seguir compilando perfectamente.

[Reto 2] Validación Perimetral DTO y Respuesta Universal

Problema en la Base: No existe validación de tipos ni reglas de negocio en tiempo de ejecución. El servidor responde con payloads planos asimétricos y expone excepciones crudas del sistema en caso de fallos.

Refactorización Exigida: Construir esquemas declarativos con **Zod** (`employee.dto.ts`) para blindar `req.body` y `req.params`. Implementar el **Response Wrapper Pattern** y un middleware interceptor global de errores para unificar las salidas HTTP exitosas y fallidas.

[Reto 3] Programación Reactiva e Inmutabilidad en el Servicio

Problema en la Base: El frontend de Angular almacena el estado en arreglos mutables locales dentro del componente de la vista y realiza peticiones directas mediante `HttpClient` en las funciones de interacción.

Refactorización Exigida: Aislar la lógica de consumo en un servicio reactivo (`EmployeeService`). Utilizar **RxJS** implementando `BehaviorSubject` privados y flujos exponenciales de solo lectura (`Observable$`) bajo el patrón de mutación de referencias inmutables (`[...current, new]`).

[Reto 4] Arquitectura de Componentes Smart vs. Dumb

Problema en la Base: El archivo HTML de la vista es un bloque monolítico masivo que mezcla el diseño de los formularios de captura con la rejilla de visualización de datos.

Refactorización Exigida: Fragmentar la UI. Crear un **Smart Component (Orquestador)** encargado de consumir los observables mediante el `async` **pipe** y delegar la renderización a **Dumb Components (Presentacionales)** independientes conectados exclusivamente por decoradores `@Input()` y `@Output()`

***Docente / Técnico Docente*:**

***Firma*:** _____